

Министерство науки и высшего образования Российской Федерации
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**
Университет ИТМО

Факультет прикладной информатики

Образовательная программа Мобильные и сетевые технологии

Лабораторная работа 6

По дисциплине Проектирование и реализация баз данных

Реализация БД с использованием СУБД MongoDB

Работа с БД в СУБД MongoDB

Выполнил: Харьковский Семён

Группа: К3341

Преподаватель: Говорова М.М.

Дата: 21.09.2026

Санкт-Петербург 2026

1 Наименование работы

Лабораторная работа 6.2. Работа с БД в СУБД MongoDB.

2 Цель работы

Овладеть практическими навыками работы с CRUD-операциями, вложенными объектами, агрегацией, изменением данных, ссылками и индексами в базе данных MongoDB.

3 Практическое задание

Создать учебную базу данных learn и выполнить практические задания методических указаний: заполнить и исследовать коллекции unicorns и towns, выполнить выборки, агрегирование и модификацию документов, создать ссылки, настроить индексы и сравнить планы выполнения запроса.

Лабораторная работа 6.1 посвящена установке и первоначальной проверке MongoDB; отчёт по ней не требуется. Индивидуальный вариант для ЛР 6.2 методическими указаниями не предусмотрен, поэтому используются заданные учебные коллекции.

4 Выполнение работы

4.1 Среда и актуальный синтаксис

Работа рассчитана на MongoDB Community Server 8.2 и оболочку mongosh. Скрипт начинается с выбора базы learn и удаления её предыдущей учебной версии, благодаря чему повторный запуск даёт одинаковое исходное состояние.

Команда методички	Команда в работе	Причина замены
insert	insertOne / insertMany	Разделение одиночной и пакетной вставки
save	insertOne / updateOne	save устарел в mongosh
update	updateOne / updateMany	Явное число изменяемых документов
remove	deleteOne / deleteMany	Актуальные операции удаления
count	countDocuments	Точный подсчёт по фильтру
ensureIndex	createIndex	Актуальный метод создания индекса

Задание 2.1.1

Формулировка: Создать БД learn, заполнить unicorns двенадцатью документами и проверить содержимое.

Команда

```
db = db.getSiblingDB("learn");
db.unicorns.insertMany([...11 документов...]);
db.unicorns.insertOne(dunx);
db.unicorns.countDocuments({});
```

Результат

```
12
{ name: 'Aurora', ... }
...
{ name: 'Unicrom', ... }
```

Вывод: База и коллекция создаются при первой вставке; в коллекции находится 12 документов.

Задание 2.2.1

Формулировка: Вывести отсортированные списки самцов и самок, ограничить список самок тремя особями и найти первую самку, любящую carrot.

Команда

```
db.unicorns.find({gender:'m'}).sort({name:1});
db.unicorns.find({gender:'f'}).sort({name:1}).limit(3);
db.unicorns.findOne({gender:'f', loves:'carrot'});
```

Результат

```
Самцы: 7 документов
Первые самки: Aurora, Ayna, Leia
Первая с carrot: Aurora
```

Вывод: Фильтрация, сортировка и ограничение курсора могут объединяться в одну цепочку.

Задание 2.2.2

Формулировка: Исключить из результата списка самцов поля пола, предпочтений и идентификатора.

Команда

```
db.unicorns.find({gender:'m'}, {_id:0, gender:0, loves:0}).sort({name:1});
```

Результат

```
{ name: 'Dunx', weight: 704, vampires: 165 }
... семь документов ...
```

Вывод: Проекция со значением 0 исключает выбранные поля из результата.

Задание 2.2.3

Формулировка: Вывести единорогов в обратном порядке добавления.

Команда

```
db.unicorns.find({}, {_id:0, name:1}).sort({$natural:-1});
```

Результат

```
Dunx, Nimue, Pilot, Leia, Raleigh, Kenny, Ayna, Solnara,
Roooooodles, Unicrom, Aurora, Horny
```

Вывод: Сортировка по `$natural:-1` возвращает документы в порядке, обратном физическому порядку вставки.

Задание 2.1.4

Формулировка: Вывести имя и первое любимое предпочтение каждого единорога без идентификатора.

Команда

```
db.unicorns.find({}, {_id:0,name:1, loves:{$slice:1}}).sort({name:1});
```

Результат

```
{ name: 'Aurora', loves: ['carrot'] }
{ name: 'Dunx', loves: ['grape'] }
...
```

Вывод: `$slice` ограничивает число элементов массива, возвращаемых в проекции.

Задание 2.3.1

Формулировка: Вывести самок весом от 500 до 700 кг без идентификатора.

Команда

```
db.unicorns.find({gender:'f',weight:{$gte:500,$lte:700}}, {_id:0});
```

Результат

```
Nimue - 540 кг
Solnara - 550 кг
Leia - 601 кг
```

Вывод: Операторы `$gte` и `$lte` задают включённый числовой диапазон.

Задание 2.3.2

Формулировка: Вывести самцов весом от 500 кг, предпочитающих одновременно `grape` и `lemon`.

Команда

```
db.unicorns.find({gender:'m',weight:{$gte:500}, loves:{$all:['grape','lemon']}}, {_id:0});
```

Результат

```
{ name: 'Kenny', loves: ['grape','lemon'], weight: 690, ... }
```

Вывод: `$all` требует наличия в массиве всех перечисленных значений.

Задание 2.3.3

Формулировка: Найти единорогов, не имеющих поля `vampires`.

Команда

```
db.unicorns.find({vampires:{$exists:false}}, {_id:0});
```

Результат

```
{ name: 'Nimue', loves: ['grape','carrot'], weight: 540, gender: 'f' }
```

Вывод: `$exists:false` выбирает документы, в которых поле отсутствует.

Задание 2.3.4

Формулировка: Вывести упорядоченный список имён самцов и первое предпочтение каждого.

Команда

```
db.unicorns.find({gender:'m'}, {_id:0, name:1, loves:{$slice:1}}).sort({name:1});
```

Результат

```
Dunx-grape; Horny-carrot; Kenny-grape; Pilot-apple;  
Raleigh-apple; Rooooooodles-apple; Unicrom-energon
```

Вывод: В одном запросе совместно применены фильтр, проекция массива и сортировка.

Задание 3.1.1

Формулировка: Создать towns; вывести города с независимыми и беспартийными мэрами.

Команда

```
db.towns.insertMany([...]);  
db.towns.find({'mayor.party':'I'}, {_id:0, name:1, mayor:1});  
db.towns.find({'mayor.party':{$exists:false}}, {_id:0, name:1, mayor:1});
```

Результат

```
Независимый мэр: New York - Michael Bloomberg  
Партия отсутствует: Punxsutawney - Jim Wehrle
```

Вывод: Точечная нотация обращается к полям вложенного документа mayor.

Задание 3.1.2

Формулировка: Сформировать функцию фильтра самцов, создать курсор первых двух имён и вывести его через forEach.

Команда

```
const maleFilter={()=>({gender:'m'})};  
const c=db.unicorns.find(maleFilter(), {_id:0, name:1}).sort({name:1}).limit(2);  
c.forEach(doc=>print(doc.name));
```

Результат

```
Dunx  
Horny
```

Вывод: Курсор вычисляется по мере перебора; sort и limit применяются до forEach.

Задание 3.2.1

Формулировка: Подсчитать самок весом от 500 до 600 кг.

Команда

```
db.unicorns.countDocuments({gender:'f',weight:{$gte:500,$lte:600}});
```

Результат

```
2
```

Вывод: Условию соответствуют Nimue и Solnara.

Задание 3.2.2

Формулировка: Вывести уникальный список предпочтений.

Команда

```
db.unicorns.distinct('loves').sort();
```

Результат

```
apple, carrot, chocolate, energon, grape, lemon, papaya,  
redbull, strawberry, sugar, watermelon
```

Вывод: distinct разворачивает значения массивов и удаляет повторения.

Задание 3.2.3

Формулировка: Посчитать количество единорогов обоих полов.

Команда

```
db.unicorns.aggregate([{$group:{_id:'$gender',count:{$sum:1}}},{ $sort: {_id:1}}]);
```

Результат

```
{ _id: 'f', count: 5 }  
{ _id: 'm', count: 7 }
```

Вывод: Стадия \$group формирует группы по полу, а \$sum подсчитывает документы.

Задание 3.3.1

Формулировка: Добавить Barny и проверить документ.

Команда

```
db.unicorns.insertOne({name:'Barny',loves:['grape'],weight:340,gender:'m'});  
db.unicorns.findOne({name:'Barny'},{_id:0});
```

Результат

```
{ name: 'Barny', loves: ['grape'], weight: 340, gender: 'm' }
```

Вывод: Вместо устаревшего save применяется insertOne.

Задание 3.3.2

Формулировка: Изменить вес Ayna на 800 кг и vampires на 51.

Команда

```
db.unicorns.updateOne({name:'Ayna'},{$set:{weight:800,vampires:51}});
```

Результат

```
matchedCount: 1, modifiedCount: 1
Ayna: weight=800, vampires=51
```

Вывод: \$set изменяет только перечисленные поля, сохраняя остальные данные документа.

Задание 3.3.3

Формулировка: Добавить Raleigh предпочтение redbull.

Команда

```
db.unicorns.updateOne({name: 'Raleigh'}, {$addToSet: {loves: 'redbull'}});
```

Результат

```
Raleigh: loves=['apple', 'sugar', 'redbull']
```

Вывод: \$addToSet предотвращает появление дубликата в массиве.

Задание 3.3.4

Формулировка: Увеличить всем самцам поле vampires на 5.

Команда

```
db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}});
```

Результат

```
matchedCount: 8, modifiedCount: 8
Horny: 68; Kenny: 44; Unicrom: 187; ...
```

Вывод: \$inc атомарно увеличивает числовое поле; у Barny отсутствующее поле создаётся со значением 5.

Задание 3.3.5

Формулировка: Удалить партийную принадлежность мэра Portland.

Команда

```
db.towns.updateOne({name: 'Portland'}, {$unset: {'mayor.party': ''}});
```

Результат

```
{ name: 'Portland', mayor: { name: 'Sam Adams' } }
```

Вывод: \$unset удаляет поле из вложенного документа.

Задание 3.3.6

Формулировка: Добавить Pilot предпочтение chocolate.

Команда

```
db.unicorns.updateOne({name: 'Pilot'}, {$addToSet: {loves: 'chocolate'}});
```

Результат

```
Pilot: loves=['apple', 'watermelon', 'chocolate']
```

Вывод: Массив расширен без замены ранее сохранённых предпочтений.

Задания 3.3.7 3.4.1 и 4.1.1

Задание 3.3.7

Добавить Aurora предпочтения sugar и lemon.

```
db.unicorns.updateOne({name:'Aurora'}, {$addToSet:{loves:{$each:['sugar','lemon']}}});
```

Результат: loves = ['carrot', 'grape', 'sugar', 'lemon']. Оператор \$each добавил несколько уникальных значений.

Задание 3.4.1

Удаление городов с беспартийными мэрами вернуло deletedCount: 2. После этого остался New York; очистка коллекции вернула deletedCount: 1.

Задание 4.1.1

Созданы зоны обитания и DBRef-ссылки. В mongosh 2.12.0 объект DBRef разыменовывается через свойства collection и oid:

```
const a = db.unicorns.findOne({name:'Aurora'});
db.getCollection(a.habitat.collection).findOne({_id:a.habitat.oid});
```

Результат фактического запуска

```
{ _id: 'forest', name: 'Лес', description: 'Лесная зона с мягким климатом' }
```

DBRef хранит пространство имён коллекции, идентификатор и при необходимости имя базы; связанный документ читается отдельным запросом.

Задания 4.2-4.4

Уникальный индекс и управление индексами

Индекс `uq_unicorn_name` создан успешно. После `dropIndexes()` остался обязательный индекс `_id_`; попытка удалить его завершилась ожидаемой ошибкой `cannot drop _id index`.

Сравнение планов запроса

Показатель	Без индекса	С индексом
Основной узел	COLLSCAN	IXSCAN
executionTimeMillis	147 мс	7 мс
totalDocsExamined	100000	4
totalKeysExamined	0	4
nReturned	4	4

Фактический запуск на MongoDB 8.2.2 подтвердил переход от полного просмотра коллекции к просмотру четырёх ключей индекса. Конкретное время зависит от компьютера, но объёмы исследованных документов и ключей показывают устойчивое преимущество индекса.

Ответы на контрольные вопросы

Как используется оператор точка

Точечная нотация обращается к вложенным полям, например `{'mayor.party':'I'}`, и к свойствам объектов JavaScript.

Как используется курсор

Курсор возвращается `find` и `aggregate`. Его перебирают через `hasNext`, `next` и `forEach`; выборку изменяют методами `sort`, `skip` и `limit`.

Какие возможности агрегирования существуют

Используются `countDocuments`, `distinct` и конвейер `aggregate` со стадиями `$match`, `$group`, `$sort`, `$project` и другими.

Что точнее настраивает изменение `save` или `update`

`updateOne` и `updateMany` изменяют отдельные поля операторами `$set`, `$inc`, `$unset`, `$addToSet` и другими. `save` устарел в `mongosh`.

Как по умолчанию удаляются документы

`deleteOne` удаляет один совпавший документ, `deleteMany` - все совпавшие. `deleteMany({})` очищает коллекцию.

Как связывают коллекции

Используются вложенные документы, ручные ссылки по `_id`, `DBRef` и объединение данных стадией `$lookup`.

Сколько индексов допустимо и как получить сведения

Одна коллекция может иметь не более 64 индексов, включая `_id_`. Метод `db.collection.getIndexes()` возвращает определения индексов коллекции.

Выводы

В базе `learn` выполнены CRUD-операции, запросы к вложенным объектам, агрегирование, изменение данных, `DBRef`-ссылки и исследование индекса. Полный сценарий успешно выполнен настоящим `mongosh 2.12.0` на `MongoDB 8.2.2`. Индекс по `value` уменьшил число исследованных документов со 100000 до 4 и время контрольного запуска со 147 до 7 мс.

Список источников и приложение

1. Методические указания. Лабораторная работа 6.1. Введение в СУБД MongoDB. Установка MongoDB.
2. Методические указания. Лабораторная работа 6.2. Работа с БД в СУБД MongoDB.
3. MongoDB Manual. CRUD Operations. <https://www.mongodb.com/docs/manual/crud/>
4. MongoDB Manual. Aggregation. <https://www.mongodb.com/docs/manual/aggregation/>
5. MongoDB Manual. Database References.
<https://www.mongodb.com/docs/manual/reference/database-references/>
6. MongoDB Manual. Indexes and explain. <https://www.mongodb.com/docs/manual/indexes/>

Приложение А Исполняемый сценарий

Файл Харьковский_ЛР6_2.js содержит исходные данные, команды всех практических заданий и вывод результатов. Команда запуска:

```
mongosh --file "Харьковский_ЛР6_2.js"
```

Контрольный запуск выполнен 02.10.2026 на MongoDB 8.2.2. Сценарий завершился с кодом 0; коллекции после выполнения: habitats, numbers, towns, unicorns. Полный текстовый лог приложен к комплекту как mongo-run.log.